

Reparación Automática de Software

Introducción

Información del curso

Teóricos - Prácticos - Laboratorios -
Clases de Consulta

- **Teóricos:**
 - Lunes 14:00 - 16:00 Lab 101
- **Prácticos - Laboratorios:**
 - Miércoles 18:00 - 21:00 Lab 101
- **Oficina:** TBD
- **Consulta:** TDB, Pueden enviarme un email o pedirme consulta cuando me vean y organizamos.
- **Email:**
sgutierrez@dc.exa.unrc.edu.ar



Información del curso

En este curso vamos a usar muchas herramientas, va a ser necesario trabajar con varios lenguajes, frameworks, y scripts. Lo más recomendable es que utilicemos Linux, bajo una arquitectura **x86-64**. Yo voy a utilizar **Ubuntu 20.04+ (64 bits version)** en WSL2, si utilizan Linux en modo dual-boot o como sistema operativo principal, pueden usar cualquier versión **20.04+** (**20.04** o mayor).

En cuanto a bibliografía, vamos a utilizar varios libros, y una gran cantidad de artículos científicos, toda la bibliografía va a estar listada en en Google Group de la materia, pueden acceder al mismo desde el link en el Google Classroom de la materia (QR de la slide anterior).

Todo material extra que vayamos necesitando, va a ser subido al Google Classroom o Google Group de la materia.

Información del curso

Presten atención a el Google Classroom y Google Group de la materia, deberían en principio recibir notificaciones, pero revisen manualmente para estar al tanto de novedades y materiales.

De qué se trata el curso?

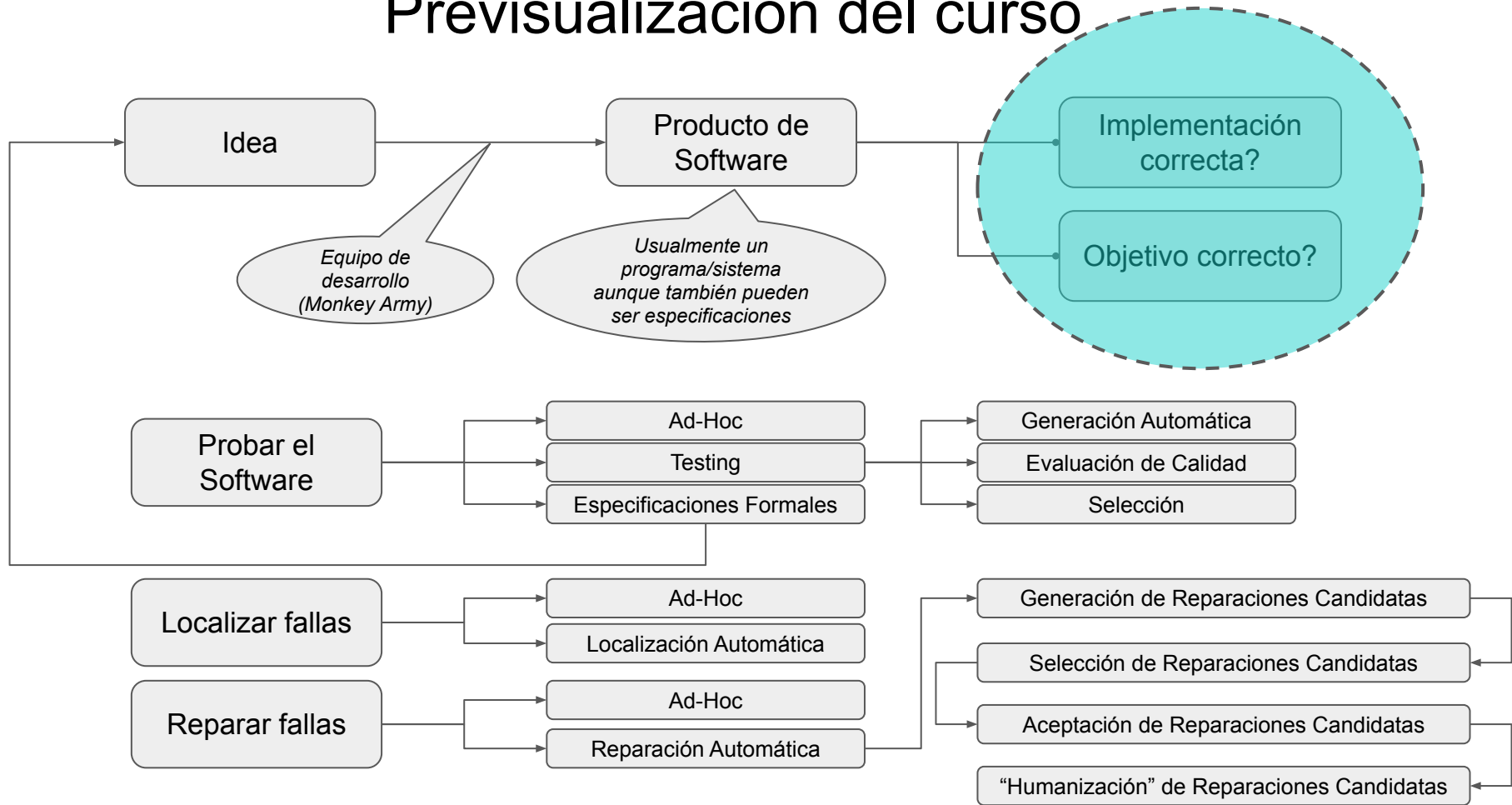
Este curso busca recorrer parte de la historia de la reparación automática de software (programa y especificaciones) y los problemas asociados, por ejemplo: localización automática de fallas. El formato estará centrado en: mostrar un problema a resolver, analizar y discutir técnicas desarrolladas, replicar y experimentar con las mismas.

El curso involucra la lectura de artículos científicos, por un lado vamos a utilizar artículos denominados "surveys" que sirven como un "estado del arte" sobre un tema particular, y por el otro lado vamos a utilizar artículos específicos de distintas técnicas.

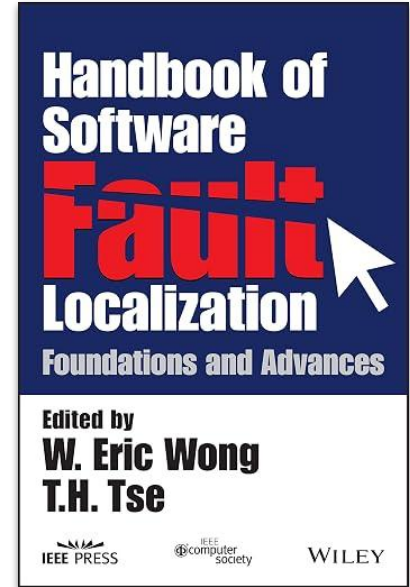
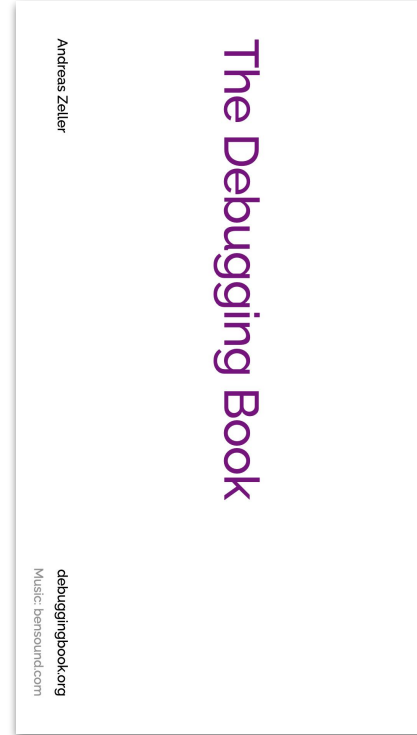
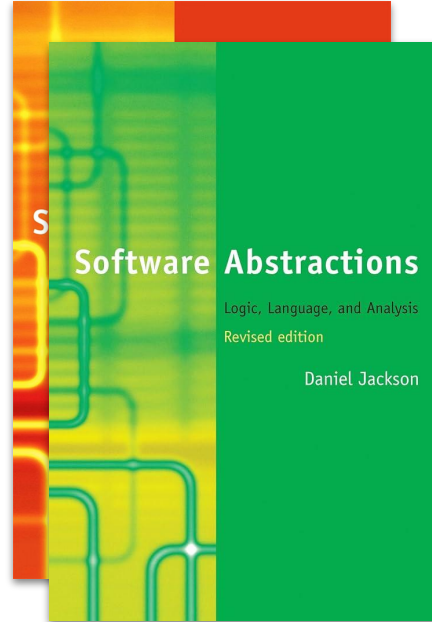
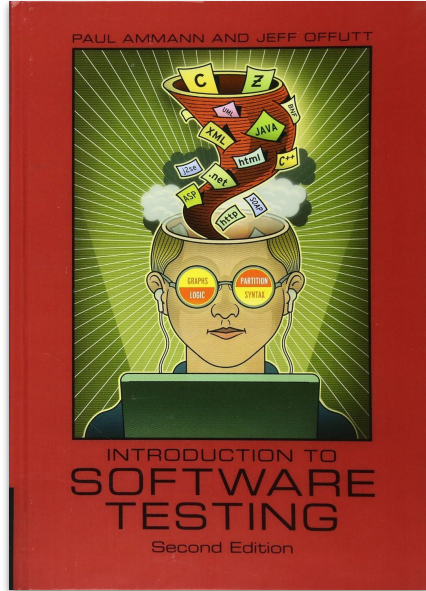
Evaluación

- Trabajos prácticos:
 - El curso tendrá varios trabajos prácticos, con nota, a realizar en grupo.
 - Los trabajos prácticos tendrán una fecha límite fija, entregas luego de esa fecha no serán aceptadas.
 - Las entregas se harán sobre una plataforma establecida y solo se aceptarán entregas en esa plataforma.
 - Si un trabajo no es entregado, la nota será cero.
 - El tamaño máximo de los grupos se determinará durante la primera semana de cursado.
- Parciales:
 - Se tomará al menos un examen parcial con su respectivo recuperatorio.
- Proyecto final:
 - El curso tendrá un proyecto final a determinar.
- Final (regular):
 - Proyecto o trabajo final a entregar y defender.
- Final (libre):
 - Examen + Proyecto o trabajo a entregar y defender.

Previsualización del curso



Bibliografía



- + Artículos científicos a determinar
- + *The Debugging Book*: debuggingbook.org
- + *Software Abstraction*, cualquier versión

Un poco de terminología inicial

Software Fault/Defecto de Software: Un defecto estático en el software, es decir, un defecto en el código u otro archivo necesario para la ejecución del software.

Software Error/Error de Software: Un estado interno incorrecto causado por un *defecto*.

Software Failure/Falla de Software: Comportamiento incorrecto del software con respecto al comportamiento esperado.

Nota: si bien es usual que estos términos se usen como sinónimos, es necesario para el contexto de esta asignatura, conocer y utilizar los términos apropiados.

Un ejemplo



```
/**  
 * Count zeros in an int array  
 */  
public static int countZeros(int[] values) {  
    int count = 0;  
    for (int i = 1; i < values.length; i++) {  
        if (values[i] == 0) {  
            count++;  
        }  
    }  
    return count;  
}
```

Un ejemplo

Software Fault/Defecto de Software

Software Error/Error de Software

Software Failure/Falla de Software

```
/**
 * Count zeros in an int array
 */
public static int countZeros(int[] values) {
    int count = 0;
    for (int i = 1; i < values.length; i++) {
        if (values[i] == 0) {
            count++;
        }
    }
    return count;
}
```

Cual es el objetivo de probar un programa?



Código correcto, problema incorrecto



Cual es el objetivo de probar un programa?



Problema correcto, implementación incorrecta



Es mi programa correcto?

Escribir software es una tarea difícil, para el cual podemos considerar los siguientes problemas:

1. Implementar un software a partir de un conjunto de requisitos escritos en lenguaje natural acarrea ambigüedades a nuestra implementación. Por ejemplo: “Implemente una función que ordene una secuencia de elementos”
 - a. ¿Que estructura usamos?
 - b. ¿Modificamos la secuencia o retornamos una nueva?
 - c. ¿Todos los elementos de la secuencia original deben aparecer en la secuencia retornada?
 - d. ¿Los elementos de la secuencia retornada deben ser exactamente los mismos que en la secuencia original?
 - e. ¿Que orden utilizamos/Cómo comparamos los elementos?
 - f. ¿Existe alguna restricción de tiempo o espacio?

Es mi programa correcto?

Escribir software es una tarea difícil, para el cual podemos considerar los siguientes problemas:

2. Errores lógicos, es fácil cometer errores lógicos al implementar un software, por ejemplo: confiar en el tamaño de una cadena de texto a leer. Además debemos considerar aspectos técnicos tales como:
 - a. Manejo manual de memoria
 - b. Representación de valores
 - c. Manejar varias condiciones que pueden cubrir más que una pequeña parte del software (por ejemplo: el estado de un objeto)

Es mi programa correcto?

Escribir software es una tarea difícil, para el cual podemos considerar los siguientes problemas:

3. Typos, de la misma forma que cometemos typos (errores de escritura) al escribir en lenguaje natural, podemos cometer los mismos tipos de errores al escribir software. Un typo puede considerarse a texto faltante, texto de más, texto incorrecto. Algunos ejemplos a mencionar son:
 - a. Usar > en lugar de <=
 - b. Olvidar un incremento/decremento/actualización en una variable asociada a la condición de un ciclo
 - c. Duplicar instrucciones

Cómo asegurar que nuestro programa es correcto?

Podemos intentar lograr un programa correcto utilizando especificaciones formales y un conjunto de reglas formales para construir el programa desde las especificaciones, nuestro programa sería correcto por construcción.

Dado un programa acompañado por especificaciones formales, podríamos hacer una evaluación formal, por ejemplo usando Lógica de Hoare para demostrar correctitud parcial o total.

Podemos ejecutar nuestro programa bajo un conjunto finito de escenarios, y evaluar si el resultado/comportamiento obtenido se corresponde con el esperado.

Cómo asegurar que nuestro programa es correcto?

Podemos intentar lograr un programa correcto utilizando especificaciones formales y un conjunto de reglas formales para construir el programa desde las especificaciones, nuestro programa sería correcto por construcción.

Si bien es posible cometer errores, es posible analizar si las reglas formales aplicadas fueron correctas. Los dos principales problemas son:

1. Necesitamos conseguir las especificaciones formales.
2. Este es un proceso difícil y que consume una gran cantidad de tiempo, solo puede ser aplicado a programas pequeños y críticos.

Cómo asegurar que nuestro programa es correcto?

Dado un programa acompañado por especificaciones formales, podríamos hacer una evaluación formal, por ejemplo usando Lógica de Hoare para demostrar correctitud parcial o total.

Si bien es posible cometer errores, es posible analizar si las reglas formales aplicadas fueron correctas. Los dos principales problemas son:

1. Necesitamos conseguir las especificaciones formales.
2. Este es un proceso difícil y que consume una gran cantidad de tiempo, solo puede ser aplicado a programas pequeños y críticos.

Cómo asegurar que nuestro programa es correcto?

Podemos ejecutar nuestro programa bajo un conjunto finito de escenarios, y evaluar si el resultado/comportamiento obtenido se corresponde con el esperado.

Por un lado, no necesitamos especificaciones formales, escribir tests no es particularmente difícil, escribir buenos tests lo es, y existen varios problemas a considerar:

1. Tenemos que lidiar con ambigüedades en especificaciones informales, es decir: lenguaje natural.
2. Un test solo puede garantizar que hay un bug, nunca que no los hay.
3. Necesitamos determinar la calidad de nuestros tests, esto determina la confianza en nuestro programa cuando no hay tests que fallan.
4. No siempre podemos correr todos los tests, y es necesario seleccionar cuales correr.

Testing 101

Arrange

Act

Assert

Testing 101

Arrange

Definir el escenario sobre el cual se quiere probar el programa. Puede incluir entradas, variables de ambiente, y cualquier cosa relacionada al estado/escenario sobre el cual se quiere ejecutar el programa.

Por ejemplo, para una función `siguiente_fecha(fecha, unidades_temporales)` debemos definir un valor para cada argumento:

```
fecha = Fecha.crear("05/03/18")  
utemp = Semanas(2)
```

Testing 101



Act

Ejecutar el programa bajo el escenario definido.

Por ejemplo para la función siguiente_fecha(fecha, unidades_temporales):

```
fecha = Fecha.crear("05/03/18")  
utemp = Semanas(2)  
nueva_fecha = siguiente_fecha(fecha, utemp)
```

Testing 101

Assert

Se evalúa el resultado/comportamiento obtenido con el esperado.

Por ejemplo, para la función `siguiente_fecha` (for example, for a function `siguiente_fecha`(fecha, unidades_temporales) tenemos:

```
fecha = Fecha.crear("05/03/18")
utemp = Semanas(2)
nueva_fecha = siguiente_fecha(fecha, utemp)
if nueva_fecha == Fecha.crear("19/03/18")
    print "Pass"
else
    print "Fail"
```

El Oráculo



Un oráculo se puede definir como una entidad que provee el comportamiento correcto/esperado que un programa debería tener cuando se ejecuta en un escenario específico.

¿Qué se puede usar como oráculo?

1. Un valor/escenario hardcodeado
2. Un archivo/base de datos relacionando escenarios específicos a sus respectivos valores/escenarios esperados
3. Una versión confiable del programa.

Un poco más de terminología

Testing¹: Evaluar un software observando su ejecución.

Test Failure/Falla de prueba²: La ejecución de un test que resulta en una **falla de software (Software Failure)**.

Debugging³: El proceso de encontrar un **defecto de software (Software Fault)** que resulta en una **falla de software (Software Failure)**.

1: Claramente traducir todos los términos no nos va a llevar lejos, la mayoría de la literatura está en inglés y se vuelve necesario usar los términos en inglés.

2: Ya está sonando un poco feo la traducción.

3: Si trato de traducir esto ya nada va a tener sentido.

¿Cuando mis tests son buenos?

Nuestros tests son buenos cuando son capaces de detectar cualquier test potencial!

¿Cuando mis tests son buenos?

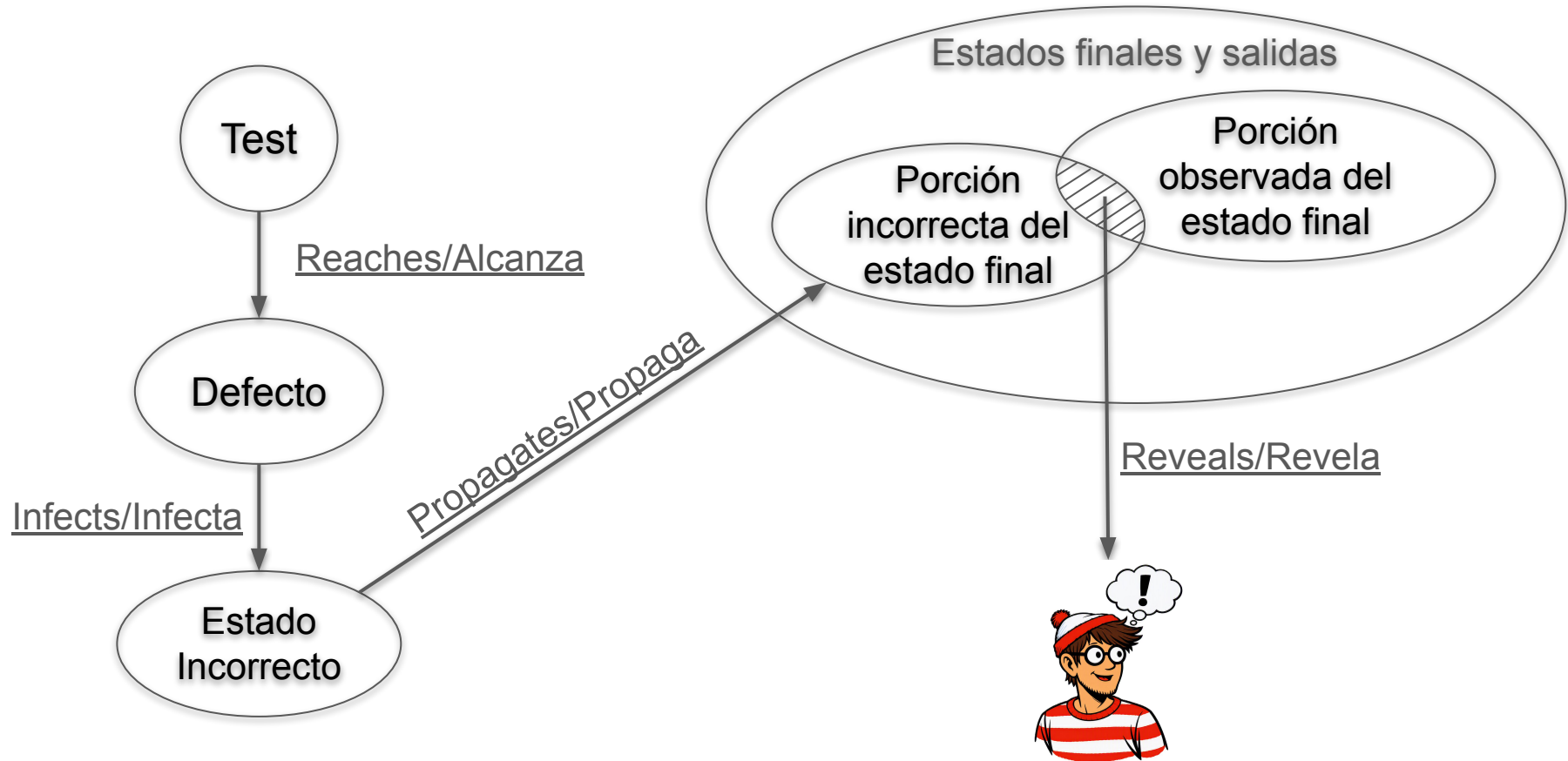
Nuestros tests son buenos cuando son capaces de detectar cualquier defecto potencial!

Al menos eso es lo que nos gustaría, pero no es posible.

- La cantidad de bugs (Software Faults) potenciales es infinita.
- La cantidad de escenarios a probar es prácticamente infinito, es decir, si bien no infinito, en la práctica es completamente imposible probar todos los escenarios posibles para un programa particular.

Lo que podemos tener son métricas indirectas sobre el potencial de nuestros tests para detectar defectos.

El modelo RIPR



Cobertura

Dada una métrica M , podemos calcular la cobertura de esa métrica por nuestros tests:

Cobertura de Sentencias: Cuántas sentencias cubren nuestros tests? Si no ejecutamos una sentencias, no podemos detectar un defecto en la misma.

Cobertura de Ramas: Cuántas ramas cubren nuestros tests? Más fuerte que cobertura de sentencias, si cubrimos todas las ramas cubrimos todas las sentencias, pero también cubrimos todas las decisiones en el código.

Cobertura de Mutantes: Cuántos bugs artificiales detectan nuestros tests? Es una buena métrica solo si los bugs artificiales son buenos representantes de bugs reales. También necesitamos una herramienta para generar estos bugs artificiales de manera automática.

Parameterized Tests

Arrange

Input

Act

Assert

Oracle

Definimos nuestros tests como funciones/métodos parametrizados con el escenario del test y el oráculo (resultado/comportamiento) esperado. Ahora el agregar, modificar, o eliminar tests se puede hacer modificando una tabla de input/output.

Parameterized Tests

Un ejemplo

```
test_empty_is_always_substring_1()  
    string_a = ""  
    string_b = "Hello"  
    result = is_substring(string_a, string_b)  
    assert result
```

```
test_is_a_substring()  
    string_a = "Good"  
    string_b = "Goodyear"  
    result = is_substring(string_a, string_b)  
    assert result
```

```
test_empty_is_always_substring_2()  
    string_a = ""  
    string_b = ""  
    result = is_substring(string_a, string_b)  
    assert result
```

```
test_is_not_a_substring()  
    string_a = "Miche"  
    string_b = "Goodyear"  
    result = is_substring(string_a, string_b)  
    assert not result
```

Parameterized Tests

Un ejemplo

```
test_is_a_substring(a, b, expected)
    result = is_substring(a, b)
    assert result == expected
```

Es muy fácil agregar, eliminar, y modificar tests, siempre que sigan las mismas instrucciones y requieran el mismo tipo de input y output.

Y si pudiéramos generar valores de manera automática?

parameters:

```
“”, “”, True
“, “Hello”, True
“Hello”, “”, False
“Ab”, “abba”, False
“ab”, “abba”, True
“bb”, “abba”, True
“ba”, “abba”, True
“aa”, “abba”, False
...
```

Property Based Testing

En lugar de definir los tests como una verificación entre el resultado obtenido y el esperado, los definimos como una propiedad que debe valer para cualquier escenario válido, por ejemplo:

$$\forall i \in \mathbb{N}_0 : \text{bin_to_int}(\text{int_to_bin}(i)) == i$$

Por supuesto que no podemos usar un test como este, si bien no es infinito, considerando los límites de representación usando enteros de 4 Bytes, tenemos 2^{32} valores posible para i .

Podemos combinar tests/propiedades como la anterior con un generador que nos va a dar un conjunto manejable de valores.

Testing Frameworks

Si bien podríamos escribir los tests “a mano” usando “print Pass” o “print Fail”, usar un framework o librería de testing nos va a ayudar a estandarizar cómo escribimos los tests, cómo evaluamos la[s] expectativa[s] en cada test, a analizar resultados, y varias características como:

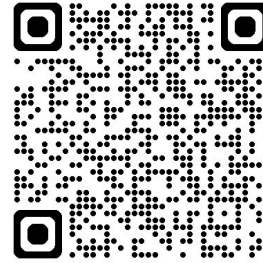
- Descubrir/Ubicar tests automáticamente, solo debemos seguir ciertas reglas y nuestros tests van a ser encontrados y ejecutados con un solo comando.
- Reportes de resultados fácil de utilizar.
- Características para definir expectativas.
- Características para definir tests que van a ser ejecutados en un conjunto de escenarios fácil de modificar.

PyTest y JUnit

Si bien existen una gran cantidad de frameworks/librerías de testing, podemos explorar dos que son conocidos y se utilizan para dos lenguajes con los cuales deberíamos estar todos cómodos de utilizar.

- PyTest para Python

- <https://docs.pytest.org/en/stable/>



- JUnit para Java

- Hay tres opciones, en orden ascendente de aparición: JUnit 4, 5, y 6
- La documentación (particularmente para 5 y 6) se basa en uso de algún Build System (Maven, Gradle, Ant, etc), nos interesa poder correr tests desde consola para aprender.

PyTest y JUnit

- JUnit para Java
 - Hay tres opciones, en orden ascendente en edad: JUnit 4, JUnit 5, y JUnit 6
 - JUnit 4 para Java 5+
(<https://github.com/junit-team/junit4/wiki/Download-and-Install>)
 - JUnit 5 para Java 8+
(<https://mvnrepository.com/artifact/org.junit.platform/junit-platform-console-standalone/1.9.3>)
 - JUnit 6 para Java 17+
(<https://mvnrepository.com/artifact/org.junit.platform/junit-platform-console-standalone/6.1.2>)

Los links son para poder ejecutar JUnit desde consola, sin ningún Build System como Maven, Gradle, Ant, etc. Esto lo vamos a usar para practicar escribir tests.

DEMO